Overview and discussion

This program uses templates to build the stacks, queues, and linked lists. The text editor can take characters or strings. It can type, undo, redo, and display the buffer. I left out the optional history to keep it simple.

Templates let me use the same class with different data types. Instead of writing separate stacks for characters and strings, I can change the type inside the angle brackets. The operations stay the same. The tests use integers for the basic structures and characters and strings for the editor.

The main thing to watch with the circular structures is the last node's next pointer. It has to point back to the first node. An empty structure and a structure with one node need their own cases. When the last node is deleted, the pointer needs to become null. Otherwise, the program could keep using a node that was already deleted. Display also needs a stopping point because a circular list never reaches a null pointer.

Arrays are straightforward and store their elements next to each other. The array stack can have unused spaces, and resizing takes O(n) time because the items have to be copied. A normal push is O(1), and push is O(1) amortized over many pushes. Pop and top are O(1). The array queue uses wraparound, so enqueue, dequeue, and front are all O(1). Its capacity is 100.

Files: structures.h contains the data structures and editor. main.cpp contains the menu and tests. The program uses only standard C++ headers.

Discussion continued

The linked stack and queue grow one node at a time, so they do not need to copy every item when they grow. Their main operations are O(1), but each node uses extra space for a pointer. Both linked lists use O(1) front insertion, rear insertion, and front deletion. Rear deletion is O(n) because the code has to find the node before the last one. Display is O(n). The linked structures use O(n) space. The array stack uses O(capacity) space, while the fixed array queue always has 100 slots.

The redo order was an important detail. A regular queue returns the first item added, but redo needs the most recently undone item. After undo, the program moves the older queue entries behind the new one. This keeps the queue's normal behavior while giving the editor the right redo order. Undo is O(r), where r is the number of items already waiting for redo. Redo is O(1). Typing is O(r) when it clears old redo items. Displaying the buffer is O(n) time and uses an extra O(n) stack to show the text in order. These costs count stored items; copying a string also depends on its length.

The array stack grows instead of rejecting the 101st item. It throws an overflow exception only if doubling would exceed the integer capacity limit. The fixed array queue throws when its 100 slots are full.

Reflection and validation

This lab helped me understand how stacks and queues work together in a program. The individual operations were easier to follow once I could see them change the text in the buffer. I also learned that getting the order right matters just as much as storing the right values.

Part 1 took 1 hour and 30 minutes. With the provided examples, it was kind of simple to put everything together. Part 2 took 1 hour because it was a short segment of this lab. Part 3 took 2 hours and 35 minutes because some outputs looked off or were flat out wrong. The total was 5 hours and 5 minutes.

Testing took the longest. Looking at the output made it easier to notice when the items were coming back in the wrong order. One thing that stood out was that undoing and redoing one item is not enough to check the editor. Doing it several times makes order problems much easier to see. The empty cases and the one-node cases also needed attention.

The automated run passed all 52 checks. The checks cover both stacks, both queues, both linked lists, and the character and string editors. They include empty operations, stack growth, queue overflow and wraparound, deleting the last node, multiple undo/redo operations, and clearing redo after new typing. The PDF includes screenshots with captions and the full source code from structures.h and main.cpp.

Validation: the code compiled with warnings treated as errors using Microsoft C++ and GCC. The Codespace run also passed with AddressSanitizer and UndefinedBehaviorSanitizer enabled. Menu checks covered characters, strings, spaces, empty strings, and invalid choices.

Stack tests

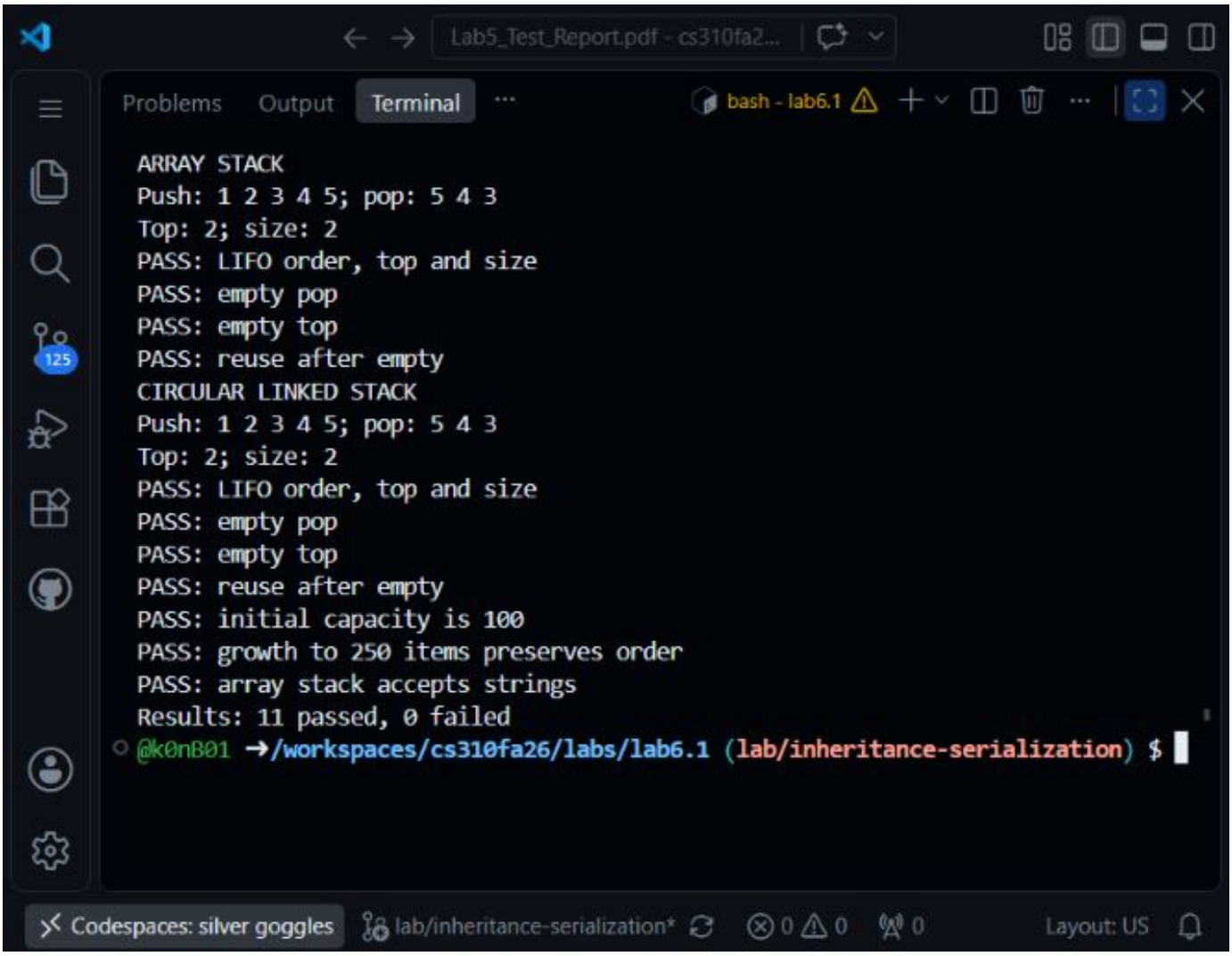


Figure 1. Both stacks pop 5, 4, and 3 after five pushes. The remaining top is 2 and the size is 2. Empty operations, reuse, array growth, and string storage pass. Result: 11 passed, 0 failed.

Captured from the program running in the GitHub Codespace. Blank lines and selected PASS lines were hidden to fit the view. The full output is saved in test_output.txt.

Queue tests

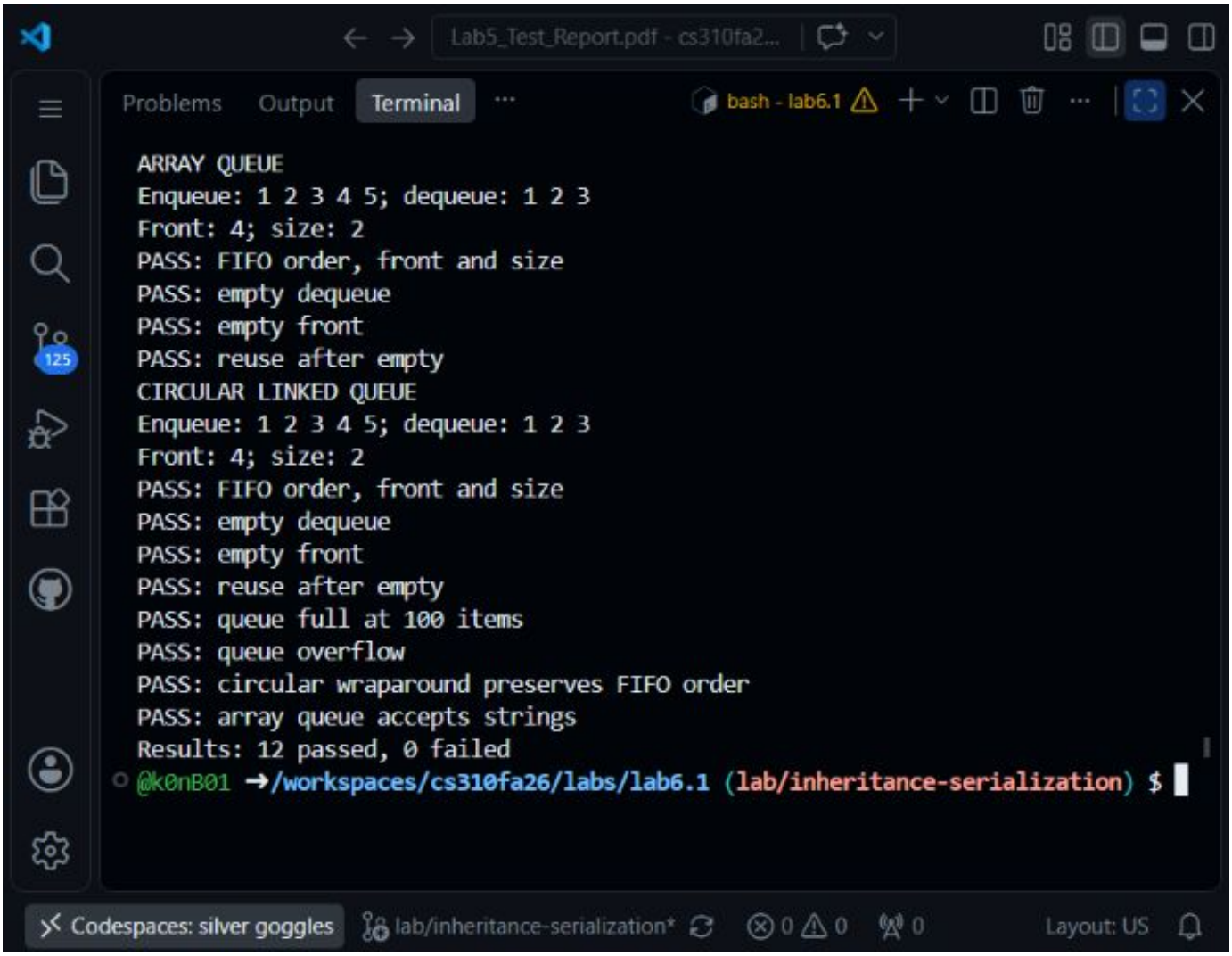


Figure 2. Both queues remove 1, 2, and 3 first. The remaining front is 4 and the size is 2. Empty operations, overflow, wraparound, reuse, and string storage pass. Result: 12 passed, 0 failed.

Captured from the program running in the GitHub Codespace. Blank lines and selected PASS lines were hidden to fit the view. The full output is saved in test_output.txt.

Linked-list tests

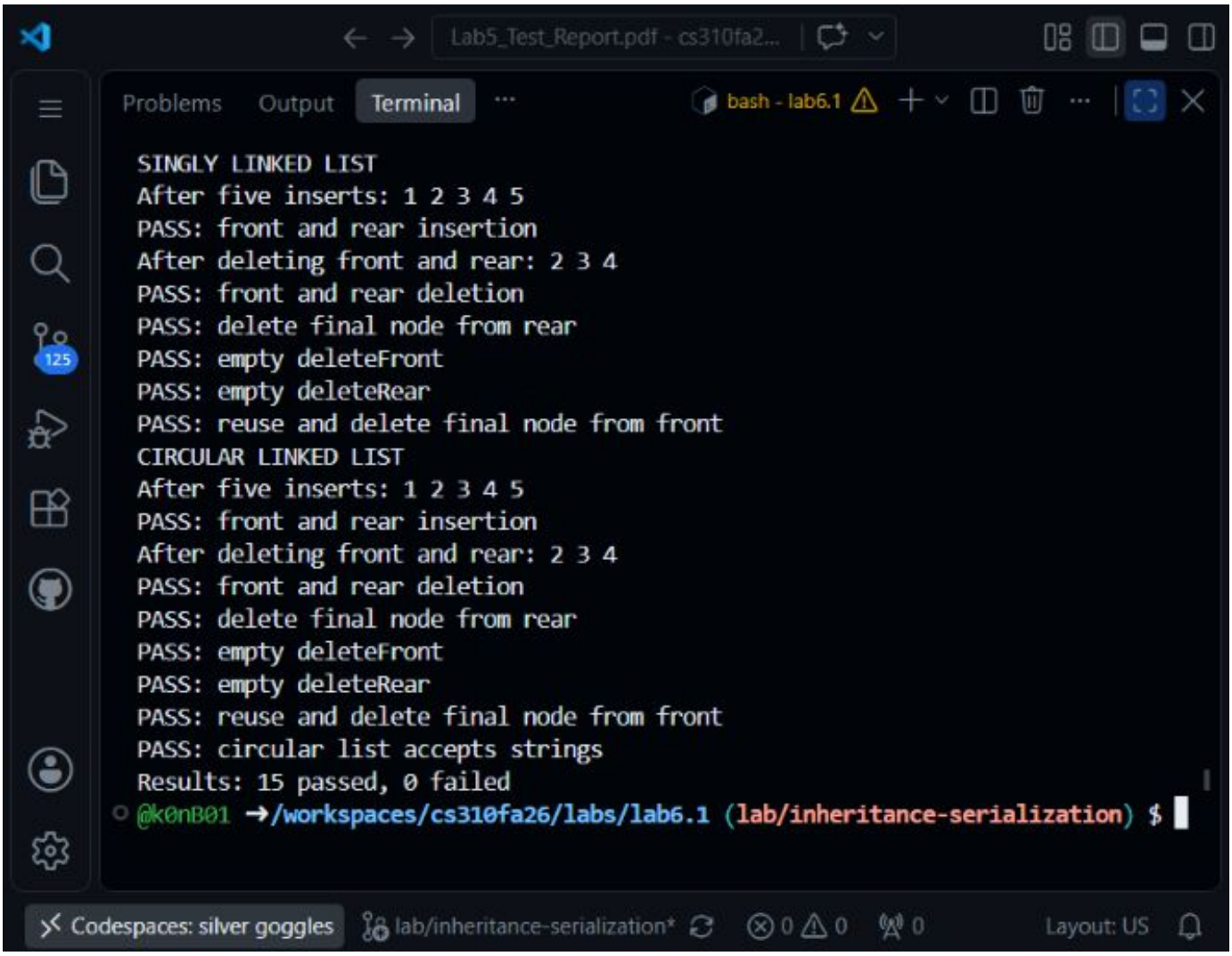


Figure 3. Both lists display 1 2 3 4 5 after insertion and 2 3 4 after deleting the front and rear. Final-node deletion and empty-operation checks pass. Result: 15 passed, 0 failed.

Captured from the program running in the GitHub Codespace. Blank lines and selected PASS lines were hidden to fit the view. The full output is saved in test_output.txt.

Text editor tests

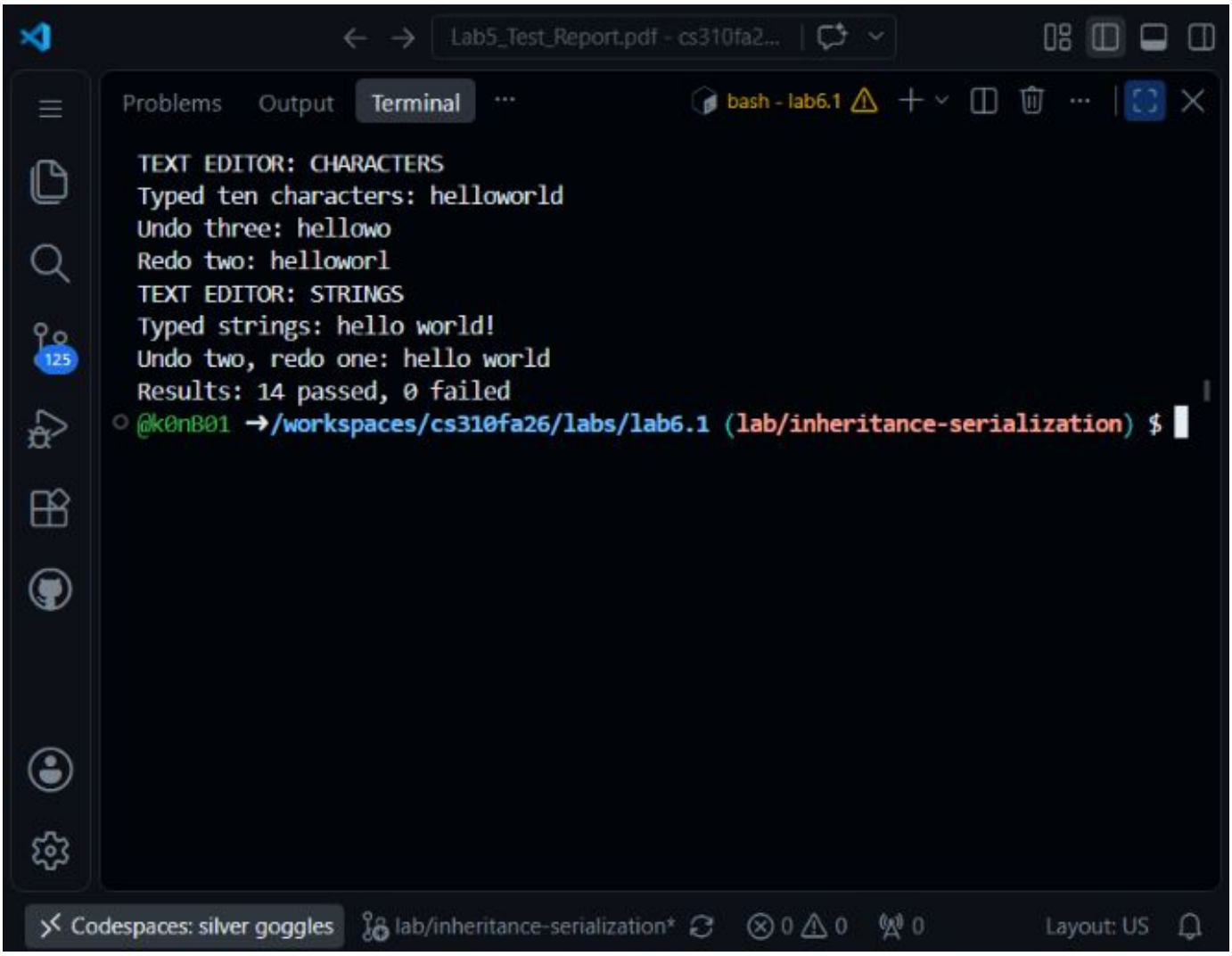


Figure 4. Ten characters become hellowo after three undos, then helloworl after two redos. String edits are undone and restored as whole items. Result: 14 passed, 0 failed.

Captured from the program running in the GitHub Codespace. Blank lines and selected PASS lines were hidden to fit the view. The full output is saved in test_output.txt.

Source code: structures.h

Lines 1-54

```
#ifndef LAB61_STRUCTURES_H
#define LAB61_STRUCTURES_H

#include <iostream>
#include <limits>
#include <stdexcept>

// Array stack: start with 100 spaces and double when full.
template <typename T>
class Stack {
    T* arr;
    int capacity;
    int count;
public:
    Stack() : arr(new T[100]), capacity(100), count(0) {}
    Stack(const Stack&) = delete;
    Stack& operator=(const Stack&) = delete;
    ~Stack() { delete[] arr; }
    bool isEmpty() const { return count == 0; }
    bool isFull() const { return count == capacity; }
    int size() const { return count; }
    void push(T item) {
        if (isFull()) {
            if (capacity > std::numeric_limits<int>::max() / 2)
                throw std::overflow_error("Stack capacity overflow");
            T* bigger = new T[capacity * 2];
            try {
                for (int i = 0; i < count; ++i) bigger[i] = arr[i];
            } catch (...) {
                delete[] bigger;
                throw;
            }
            delete[] arr;
            arr = bigger;
            capacity *= 2;
        }
        arr[count] = item;
        ++count;
    }
    T pop() {
        if (isEmpty()) throw std::underflow_error("Stack is empty");
        T item = arr[count - 1];
        --count;
        return item;
    }
    T top() const {
        if (isEmpty()) throw std::underflow_error("Stack is empty");
        return arr[count - 1];
    }
};

// The top is rear->next. The rear points back to the top.
template <typename T>
class CircularLinkedStack {
```

Source code: structures.h

Lines 55-108

```
55      struct Node {
56          T data;
57          Node* next;
58          Node(T item) : data(item), next(nullptr) {}
59      };
60      Node* rear;
61      int count;
62  public:
63      CircularLinkedStack() : rear(nullptr), count(0) {}
64      CircularLinkedStack(const CircularLinkedStack&) = delete;
65      CircularLinkedStack& operator=(const CircularLinkedStack&) = delete;
66      ~CircularLinkedStack() {
67          while (rear) {
68              Node* old = rear->next;
69              if (old == rear) rear = nullptr;
70              else rear->next = old->next;
71              delete old;
72          }
73      }
74      bool isEmpty() const { return count == 0; }
75      int size() const { return count; }
76      void push(T item) {
77          Node* node = new Node(item);
78          if (isEmpty()) {
79              rear = node;
80              node->next = node;
81          } else {
82              node->next = rear->next;
83              rear->next = node;
84          }
85          ++count;
86      }
87      T pop() {
88          if (isEmpty()) throw std::underflow_error("Stack is empty");
89          Node* old = rear->next;
90          T item = old->data;
91          if (old == rear) rear = nullptr;
92          else rear->next = old->next;
93          delete old;
94          --count;
95          return item;
96      }
97      T top() const {
98          if (isEmpty()) throw std::underflow_error("Stack is empty");
99          return rear->next->data;
100     }
101     void display(std::ostream& out = std::cout) const {
102         Stack<T> reverse;
103         if (rear) {
104             Node* current = rear->next;
105             for (int i = 0; i < count; ++i) {
106                 reverse.push(current->data);
107                 current = current->next;
108             }
```

Source code: structures.h

Lines 109-162

```
109         }
110         while (!reverse.isEmpty()) out << reverse.pop();
111     }
112 };
113 
114 // Fixed-capacity circular array queue, as required by the lab.
115 template <typename T>
116 class Queue {
117     T* arr;
118     int capacity;
119     int frontIndex;
120     int rearIndex;
121     int count;
122 public:
123     Queue() : arr(new T[100]), capacity(100), frontIndex(0),
124               rearIndex(-1), count(0) {}
125     Queue(const Queue&) = delete;
126     Queue& operator=(const Queue&) = delete;
127     ~Queue() { delete[] arr; }
128     bool isEmpty() const { return count == 0; }
129     bool isFull() const { return count == capacity; }
130     int size() const { return count; }
131     void enqueue(T item) {
132         if (isFull()) throw std::overflow_error("Queue is full");
133         int next = (rearIndex + 1) % capacity;
134         arr[next] = item;
135         rearIndex = next;
136         ++count;
137     }
138     T dequeue() {
139         if (isEmpty()) throw std::underflow_error("Queue is empty");
140         T item = arr[frontIndex];
141         frontIndex = (frontIndex + 1) % capacity;
142         --count;
143         return item;
144     }
145     T front() const {
146         if (isEmpty()) throw std::underflow_error("Queue is empty");
147         return arr[frontIndex];
148     }
149 };
150 
151 template <typename T>
152 class CircularLinkedQueue {
153     struct Node {
154         T data;
155         Node* next;
156         Node(T item) : data(item), next(nullptr) {}
157     };
158     Node* rear;
159     int count;
160 public:
161     CircularLinkedQueue() : rear(nullptr), count(0) {}
162     CircularLinkedQueue(const CircularLinkedQueue&) = delete;
```

Source code: structures.h

Lines 163-216

```
163     CircularLinkedQueue& operator=(const CircularLinkedQueue&) = delete;
164     ~CircularLinkedQueue() {
165         while (rear) {
166             Node* old = rear->next;
167             if (old == rear) rear = nullptr;
168             else rear->next = old->next;
169             delete old;
170         }
171     }
172     bool isEmpty() const { return count == 0; }
173     int size() const { return count; }
174     void enqueue(T item) {
175         Node* node = new Node(item);
176         if (isEmpty()) node->next = node;
177         else {
178             node->next = rear->next;
179             rear->next = node;
180         }
181         rear = node;
182         ++count;
183     }
184     T dequeue() {
185         if (isEmpty()) throw std::underflow_error("Queue is empty");
186         Node* old = rear->next;
187         T item = old->data;
188         if (old == rear) rear = nullptr;
189         else rear->next = old->next;
190         delete old;
191         --count;
192         return item;
193     }
194     T front() const {
195         if (isEmpty()) throw std::underflow_error("Queue is empty");
196         return rear->next->data;
197     }
198 };
199 
200 template <typename T>
201 class SinglyLinkedList {
202     struct Node {
203         T data;
204         Node* next;
205         Node(T item) : data(item), next(nullptr) {}
206     };
207     Node* head;
208     Node* tail;
209     int count;
210 public:
211     SinglyLinkedList() : head(nullptr), tail(nullptr), count(0) {}
212     SinglyLinkedList(const SinglyLinkedList&) = delete;
213     SinglyLinkedList& operator=(const SinglyLinkedList&) = delete;
214     ~SinglyLinkedList() { while (!isEmpty()) deleteFront(); }
215     bool isEmpty() const { return count == 0; }
216     int size() const { return count; }
```

Source code: structures.h

Lines 217-270

```
217     void insertFront(T item) {
218         Node* node = new Node(item);
219         node->next = head;
220         head = node;
221         if (!tail) tail = node;
222         ++count;
223     }
224     void insertRear(T item) {
225         Node* node = new Node(item);
226         if (tail) tail->next = node;
227         else head = node;
228         tail = node;
229         ++count;
230     }
231     void deleteFront() {
232         if (isEmpty()) throw std::underflow_error("List is empty");
233         Node* old = head;
234         head = head->next;
235         delete old;
236         if (!head) tail = nullptr;
237         --count;
238     }
239     void deleteRear() {
240         if (isEmpty()) throw std::underflow_error("List is empty");
241         if (head == tail) { deleteFront(); return; }
242         Node* current = head;
243         while (current->next != tail) current = current->next;
244         delete tail;
245         tail = current;
246         tail->next = nullptr;
247         --count;
248     }
249     void display(std::ostream& out = std::cout) const {
250         for (Node* current = head; current; current = current->next)
251             out << current->data << ' ';
252         out << '\n';
253     }
254 };
255 
256 template <typename T>
257 class CircularLinkedList {
258     struct Node {
259         T data;
260         Node* next;
261         Node(T item) : data(item), next(nullptr) {}
262     };
263     Node* rear;
264     int count;
265 public:
266     CircularLinkedList() : rear(nullptr), count(0) {}
267     CircularLinkedList(const CircularLinkedList&) = delete;
268     CircularLinkedList& operator=(const CircularLinkedList&) = delete;
269     ~CircularLinkedList() { while (!isEmpty()) deleteFront(); }
270     bool isEmpty() const { return count == 0; }
```

Source code: structures.h

Lines 271-324

```
271     int size() const { return count; }
272     void insertFront(T item) {
273         Node* node = new Node(item);
274         if (isEmpty()) {
275             rear = node;
276             node->next = node;
277         } else {
278             node->next = rear->next;
279             rear->next = node;
280         }
281         ++count;
282     }
283     void insertRear(T item) {
284         insertFront(item);
285         rear = rear->next;
286     }
287     void deleteFront() {
288         if (isEmpty()) throw std::underflow_error("List is empty");
289         Node* old = rear->next;
290         if (old == rear) rear = nullptr;
291         else rear->next = old->next;
292         delete old;
293         --count;
294     }
295     void deleteRear() {
296         if (isEmpty()) throw std::underflow_error("List is empty");
297         if (count == 1) { deleteFront(); return; }
298         Node* current = rear->next;
299         while (current->next != rear) current = current->next;
300         current->next = rear->next;
301         delete rear;
302         rear = current;
303         --count;
304     }
305     void display(std::ostream& out = std::cout) const {
306         if (rear) {
307             Node* current = rear->next;
308             for (int i = 0; i < count; ++i) {
309                 out << current->data << ' ';
310                 current = current->next;
311             }
312         }
313         out << '\n';
314     }
315 };
316 
317 template <typename T>
318 class TextEditorBuffer {
319     CircularLinkedStack<T> buffer;
320     CircularLinkedQueue<T> redoQueue;
321 public:
322     void type(T item) {
323         buffer.push(item);
324         // A new edit clears the old redo path.
```

Source code: structures.h

Lines 325-349

```
325          while (!redoQueue.isEmpty()) redoQueue.dequeue();
326      }
327      void undo() {
328          if (buffer.isEmpty())
329              throw std::underflow_error("Nothing to undo");
330          int oldCount = redoQueue.size();
331          redoQueue.enqueue(buffer.top());
332          buffer.pop();
333          // Move older undone items behind the newest undone item.
334          for (int i = 0; i < oldCount; ++i)
335              redoQueue.enqueue(redoQueue.dequeue());
336      }
337      void redo() {
338          if (redoQueue.isEmpty())
339              throw std::underflow_error("Nothing to redo");
340          buffer.push(redoQueue.front());
341          redoQueue.dequeue();
342      }
343      void displayBuffer(std::ostream& out = std::cout) const {
344          buffer.display(out);
345          out << '\n';
346      }
347  };
348
349  #endif
```

Source code: main.cpp

Lines 1-54

```
#include "structures.h"
#include <sstream>
#include <string>

int passed = 0;
int failed = 0;

void check(bool condition, const std::string& name) {
    std::cout << (condition ? "PASS: " : "FAIL: ") << name << '\n';
    if (condition) ++passed;
    else ++failed;
}

template <typename Exception, typename Action>
void checkThrows(Action action, const std::string& name) {
    bool caught = false;
    try { action(); }
    catch (const Exception&) { caught = true; }
    catch (...) {}
    check(caught, name);
}

template <typename S>
void testStack(const std::string& name) {
    std::cout << "\n" << name << '\n';
    S stack;
    for (int i = 1; i <= 5; ++i) stack.push(i);
    bool order = stack.pop() == 5;
    order = (stack.pop() == 4) && order;
    order = (stack.pop() == 3) && order;
    std::cout << "Push: 1 2 3 4 5; pop: 5 4 3\n";
    std::cout << "Top: " << stack.top() << "; size: " << stack.size() << '\n';
    check(order && stack.top() == 2 && stack.size() == 2,
          "LIFO order, top and size");
    stack.pop(); stack.pop();
    checkThrows<std::underflow_error>([&]() { stack.pop(); }, "empty pop");
    checkThrows<std::underflow_error>([&]() { stack.top(); }, "empty top");
    stack.push(9);
    check(stack.pop() == 9 && stack.isEmpty(), "reuse after empty");
}

void testStacks() {
    testStack<Stack<int>>("ARRAY STACK");
    testStack<CircularLinkedStack<int>>("CIRCULAR LINKED STACK");
    Stack<int> stack;
    for (int i = 0; i < 100; ++i) stack.push(i);
    check(stack.isFull(), "initial capacity is 100");
    for (int i = 100; i < 250; ++i) stack.push(i);
    bool order = stack.size() == 250 && !stack.isFull();
    for (int i = 249; i >= 0; --i) order = (stack.pop() == i) && order;
    check(order && stack.isEmpty(), "growth to 250 items preserves order");
    Stack<std::string> words;
    words.push("hello");
    check(words.pop() == "hello", "array stack accepts strings");
```

Source code: main.cpp

Lines 55-108

```
55  }
56
57  template <typename Q>
58  void testQueue(const std::string& name) {
59      std::cout << "\n" << name << '\n';
60      Q queue;
61      for (int i = 1; i <= 5; ++i) queue.enqueue(i);
62      bool order = queue.dequeue() == 1;
63      order = (queue.dequeue() == 2) && order;
64      order = (queue.dequeue() == 3) && order;
65      std::cout << "Enqueue: 1 2 3 4 5; dequeue: 1 2 3\n";
66      std::cout << "Front: " << queue.front() << "; size: " << queue.size() << '\n';
67      check(order && queue.front() == 4 && queue.size() == 2,
68            "FIFO order, front and size");
69      queue.dequeue(); queue.dequeue();
70      checkThrows<std::underflow_error>([&]() { queue.dequeue(); }, "empty dequeue");
71      checkThrows<std::underflow_error>([&]() { queue.front(); }, "empty front");
72      queue.enqueue(9);
73      check(queue.dequeue() == 9 && queue.isEmpty(), "reuse after empty");
74  }
75
76  void testQueues() {
77      testQueue<Queue<int>>("ARRAY QUEUE");
78      testQueue<CircularLinkedQueue<int>>("CIRCULAR LINKED QUEUE");
79      Queue<int> queue;
80      for (int i = 0; i < 100; ++i) queue.enqueue(i);
81      check(queue.isFull(), "queue full at 100 items");
82      checkThrows<std::overflow_error>([&]() { queue.enqueue(100); }, "queue overflow");
83      bool order = queue.size() == 100 && queue.front() == 0;
84      for (int i = 0; i < 60; ++i) order = (queue.dequeue() == i) && order;
85      for (int i = 100; i < 160; ++i) queue.enqueue(i);
86      for (int i = 60; i < 160; ++i) order = (queue.dequeue() == i) && order;
87      check(order && queue.isEmpty(), "circular wraparound preserves FIFO order");
88      Queue<std::string> words;
89      words.enqueue("hello");
90      check(words.dequeue() == "hello", "array queue accepts strings");
91  }
92
93  template <typename L>
94  void testList(const std::string& name) {
95      std::cout << "\n" << name << '\n';
96      L list;
97      list.insertRear(3); list.insertFront(2); list.insertFront(1);
98      list.insertRear(4); list.insertRear(5);
99      std::cout << "After five inserts: "; list.display();
100     std::ostringstream before;
101     list.display(before);
102     check(before.str() == "1 2 3 4 5 \n" && list.size() == 5,
103           "front and rear insertion");
104     list.deleteFront(); list.deleteRear();
105     std::cout << "After deleting front and rear: "; list.display();
106     std::ostringstream after;
107     list.display(after);
108     check(after.str() == "2 3 4 \n" && list.size() == 3,
```

Source code: main.cpp

Lines 109-162

```
109           "front and rear deletion");
110     list.deleteRear(); list.deleteFront(); list.deleteRear();
111     check(list.isEmpty(), "delete final node from rear");
112     checkThrows<std::underflow_error>([&]() { list.deleteFront(); }, "empty deleteFront");
113     checkThrows<std::underflow_error>([&]() { list.deleteRear(); }, "empty deleteRear");
114     list.insertFront(9); list.deleteFront();
115     check(list.isEmpty(), "reuse and delete final node from front");
116     std::ostringstream empty;
117     list.display(empty);
118     check(empty.str() == "\n", "empty display");
119 }
120 
121 void testLists() {
122     testList<SinglyLinkedList<int>>("SINGLY LINKED LIST");
123     testList<CircularLinkedList<int>>("CIRCULAR LINKED LIST");
124     CircularLinkedList<std::string> words;
125     words.insertRear("hello"); words.insertRear("world");
126     std::ostringstream out;
127     words.display(out);
128     check(out.str() == "hello world \n", "circular list accepts strings");
129 }
130 
131 template <typename T>
132 std::string contents(const TextEditorBuffer<T>& editor) {
133     std::ostringstream out;
134     editor.displayBuffer(out);
135     return out.str();
136 }
137 
138 void testEditor() {
139     std::cout << "\nTEXT EDITOR: CHARACTERS\n";
140     TextEditorBuffer<char> editor;
141     checkThrows<std::underflow_error>([&]() { editor.undo(); }, "empty undo");
142     checkThrows<std::underflow_error>([&]() { editor.redo(); }, "empty redo");
143     // Exactly ten characters; "hello world" would be eleven.
144     for (char ch : std::string("helloworld")) editor.type(ch);
145     std::cout << "Typed ten characters: "; editor.displayBuffer();
146     check(contents(editor) == "helloworld\n", "ten-character input");
147     editor.undo(); editor.undo(); editor.undo();
148     std::cout << "Undo three: "; editor.displayBuffer();
149     check(contents(editor) == "hellowo\n", "undo three characters");
150     editor.redo(); editor.redo();
151     std::cout << "Redo two: "; editor.displayBuffer();
152     check(contents(editor) == "helloworl\n", "redo newest undone item first");
153     editor.redo();
154     check(contents(editor) == "helloworld\n", "restore original text");
155     editor.undo(); editor.type('!');
156     checkThrows<std::underflow_error>([&]() { editor.redo(); }, "new typing clears redo");
157     check(contents(editor) == "helloworl!\n", "new edit stays in buffer");
158     for (int i = 0; i < 10; ++i) editor.undo();
159     check(contents(editor) == "\n", "undo all items");
160     for (int i = 0; i < 10; ++i) editor.redo();
161     check(contents(editor) == "helloworl!\n", "redo all items in order");
162     editor.undo(); editor.redo(); editor.undo(); editor.redo();
```

Source code: main.cpp

Lines 163-216

```
    check(contents(editor) == "helloworl!\n", "alternating undo and redo");

    std::cout << "\nTEXT EDITOR: STRINGS\n";
    TextEditorBuffer<std::string> words;
    words.type("hello "); words.type("world"); words.type("!");
    std::cout << "Typed strings: "; words.displayBuffer();
    check(contents(words) == "hello world!\n", "string input");
    words.undo(); words.undo(); words.redo();
    std::cout << "Undo two, redo one: "; words.displayBuffer();
    check(contents(words) == "hello world\n", "whole-string undo and redo");
    words.redo();
    check(contents(words) == "hello world!\n", "restore all strings");
}

void setInput(char& item, const std::string& input) { item = input[0]; }
void setInput(std::string& item, const std::string& input) { item = input; }

template <typename T>
void menu(bool characterMode) {
    TextEditorBuffer<T> editor;
    std::string choice;
    while (true) {
        std::cout << "\nText Editor Buffer Simulation\n"
                  << "1. Type\n2. Undo\n3. Redo\n4. Display Buffer\n5. Exit\n"
                  << "Enter choice: ";
        if (!std::getline(std::cin, choice) || choice == "5") return;
        try {
            if (choice == "1") {
                std::string input;
                std::cout << (characterMode ? "Enter one character: " : "Enter a string: ");
                if (!std::getline(std::cin, input)) return;
                if (characterMode && input.size() != 1) {
                    std::cout << "Please enter exactly one character.\n";
                    continue;
                }
                T item;
                setInput(item, input);
                editor.type(item);
            } else if (choice == "2") editor.undo();
            else if (choice == "3") editor.redo();
            else if (choice != "4") {
                std::cout << "Please choose 1 through 5.\n";
                continue;
            }
            std::cout << "Buffer: "; editor.displayBuffer();
        } catch (const std::exception& error) {
            std::cout << error.what() << '\n';
        }
    }
}

int main(int argc, char* argv[]) {
    if (argc > 1 && std::string(argv[1]) == "--test") {
        std::string group = argc > 2 ? argv[2] : "all";
```

Source code: main.cpp

Lines 217-241

```
217         if (group != "all" && group != "stacks" && group != "queues"
218             && group != "lists" && group != "editor") {
219             std::cerr << "Unknown test group\n";
220             return 1;
221         }
222         try {
223             if (group == "all" || group == "stacks") testStacks();
224             if (group == "all" || group == "queues") testQueues();
225             if (group == "all" || group == "lists") testLists();
226             if (group == "all" || group == "editor") testEditor();
227         } catch (const std::exception& error) {
228             std::cout << "FAIL: unexpected exception: " << error.what() << '\n';
229             ++failed;
230         }
231         std::cout << "\nResults: " << passed << " passed, " << failed << " failed\n";
232         return failed == 0 ? 0 : 1;
233     }
234     std::string mode;
235     std::cout << "Choose input mode (1 = characters, 2 = strings): ";
236     if (!std::getline(std::cin, mode)) return 0;
237     if (mode == "1") menu<char>(true);
238     else if (mode == "2") menu<std::string>(false);
239     else { std::cout << "Invalid mode.\n"; return 1; }
240     return 0;
241 }
```